

CAT 2 PENETRATION TESTING LAB REPORT :KIOPTRIX 2

GROUP F

MEMBERS

1. 24/00126 Warui Peter Willis
2. 24/00023 Ochieng David
3. 24/00233 Margaret Nyambura Chuchu
4. 24/02395 Muiruri Victoria Nyanjau
5. 24/08462 Florence Rotich
6. 21/04447 Victor Maingi
7. 23/04384 Lianne Wanjira

1. Executive Summary

This report details the systematic penetration testing process conducted against the Kioptrix Level 2 vulnerable virtual machine. The assessment covers host discovery, service enumeration, web application exploitation via SQL Injection (SQLi) and Command Injection, initial shell access, and final local privilege escalation to achieve full administrative control.

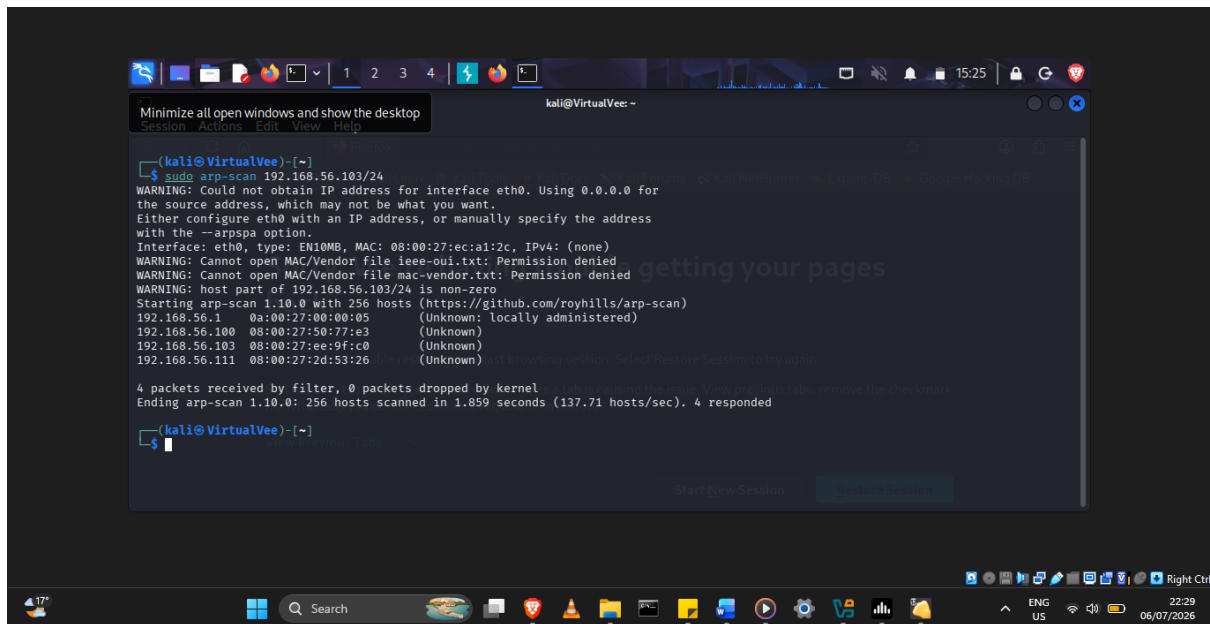
2. Reconnaissance & Network Scanning

Host Discovery

To map active hosts within the isolated host-only network layout, the arp-scan tool was executed from the attacking machine

```
sudo arp-scan 192.168.56.103/24
```

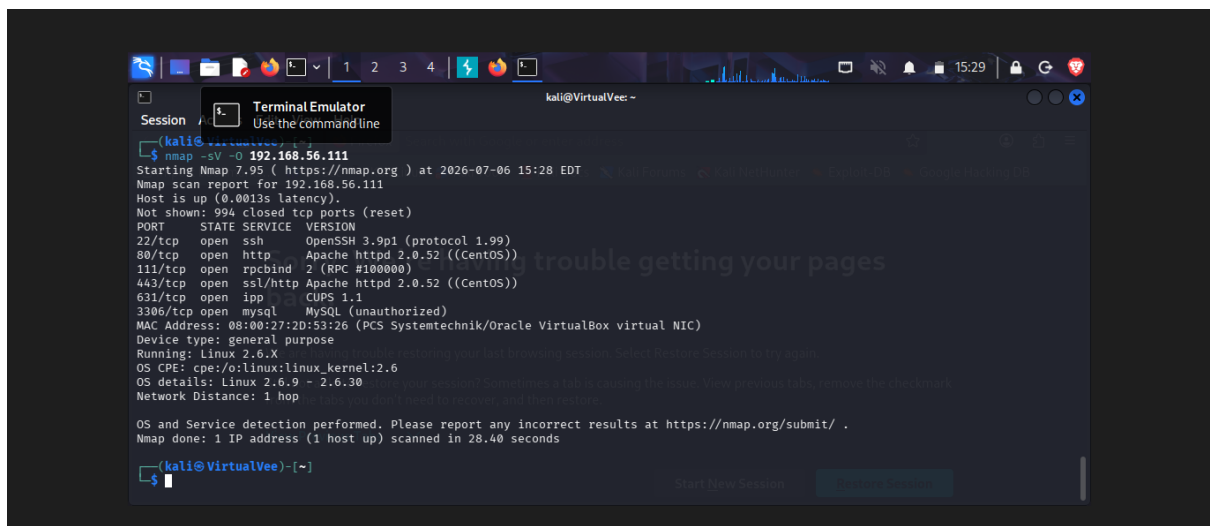
Target ip 192.168.56.111

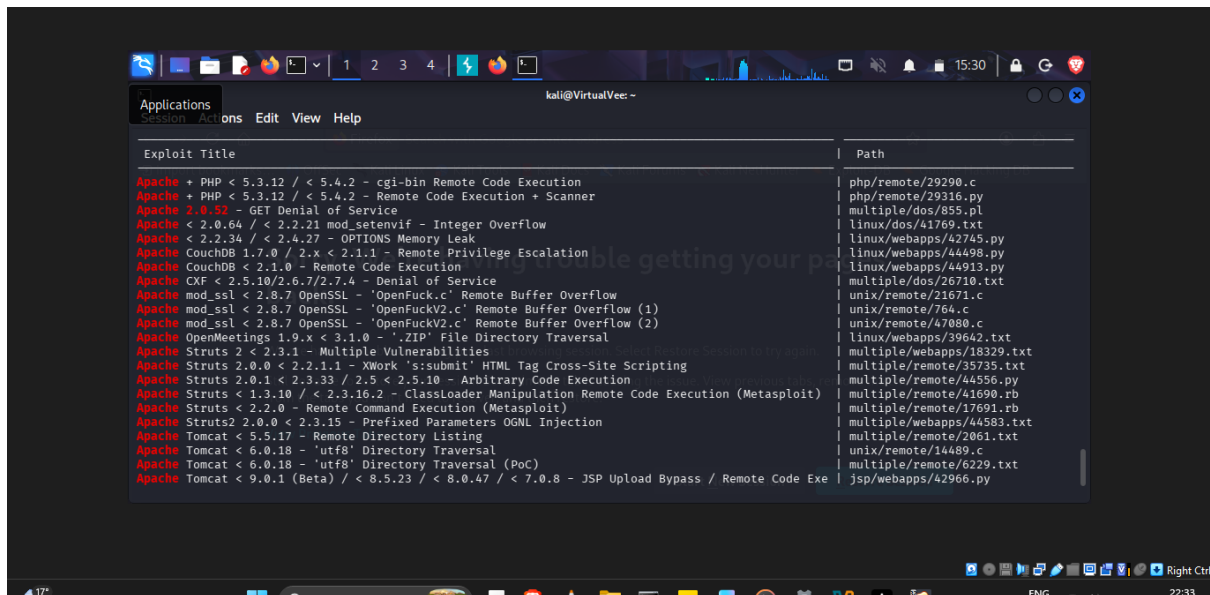


Port Scanning & Service Enumeration

An aggressive network scan was performed using nmap to pinpoint active services and fingerprint the underlying operating system:

`nmap -sV -O 192.168.56.111`





Web Application Exploitation & Initial Access

Vulnerability Research

A standard query using searchsploit for **Apache 2.0.52** indicated multiple known legacy buffer overflows, Denial of Service vectors, and module flaws. However, flaws within the hosted web application layer provided a more direct path to exploitation.

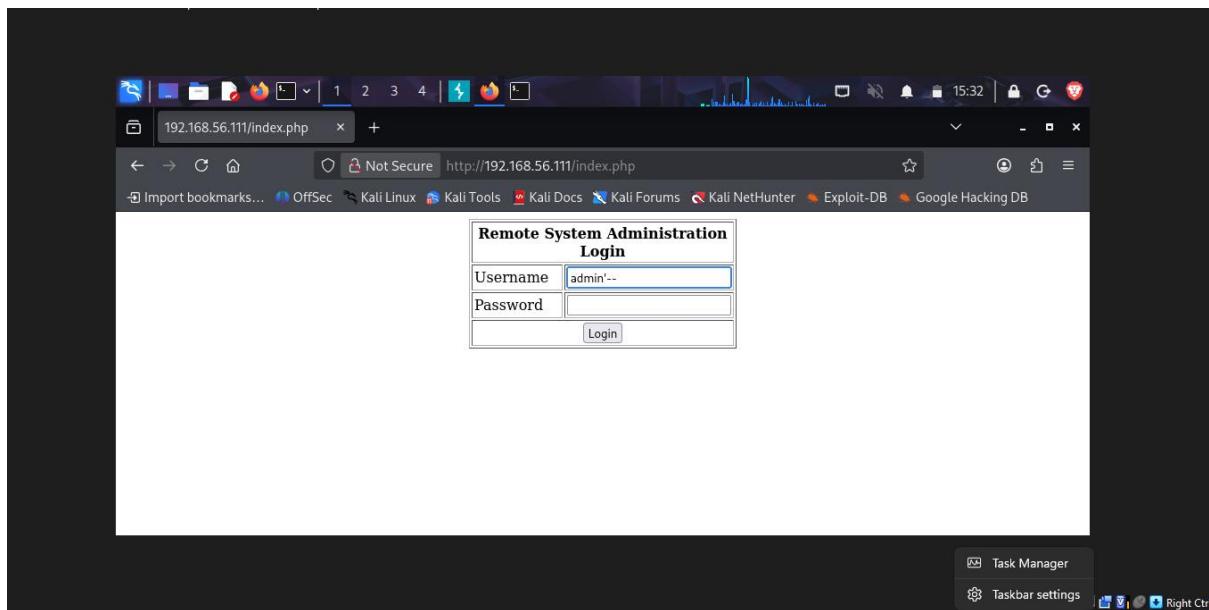
Authentication Bypass (SQL Injection)

Navigating to the web root

(http://192.168.56.111/index.php) presented a

Remote System Administration Login interface.

- **Exploitation Technique:** A standard SQL injection payload was entered into the username field
- Username: admin'--
- Password:



Result: The application failed to sanitize the input, short-circuiting the SQL query logic and successfully granting unauthorized access to the internal web console without requiring valid credentials.

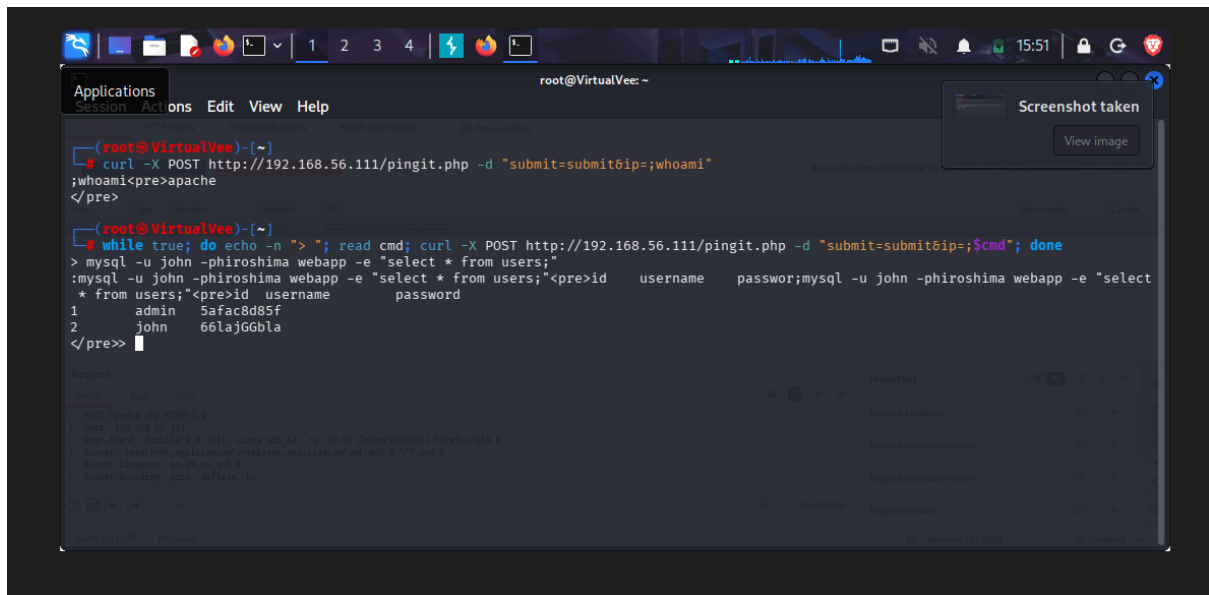
Command Injection Discovery

Once logged into the **Basic Administrative Web Console**, a feature titled "**Ping a Machine on the Network**" was observed.

- **Testing:** Inputting a valid local target IP (192.168.56.102) returned typical automated system ICMP output.
- **Exploitation:** Because the input field fed directly into an underlying shell execution command without sanitization, malicious commands could be chained using a semicolon separator (;):

192.168.56.102;ls;id;whoami

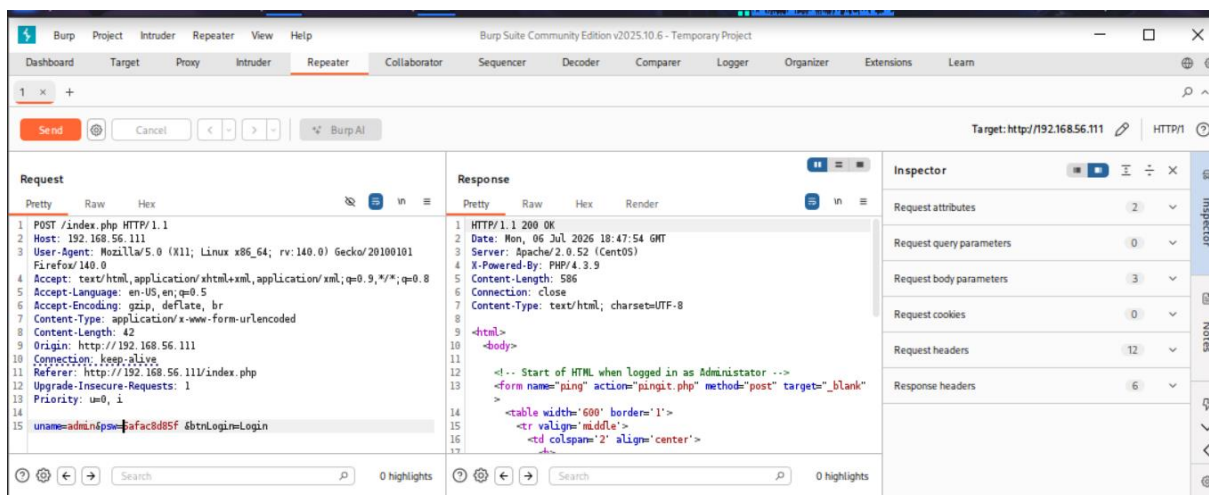
Result: The web server executed the instructions, outputting the runtime user identity: Apache.



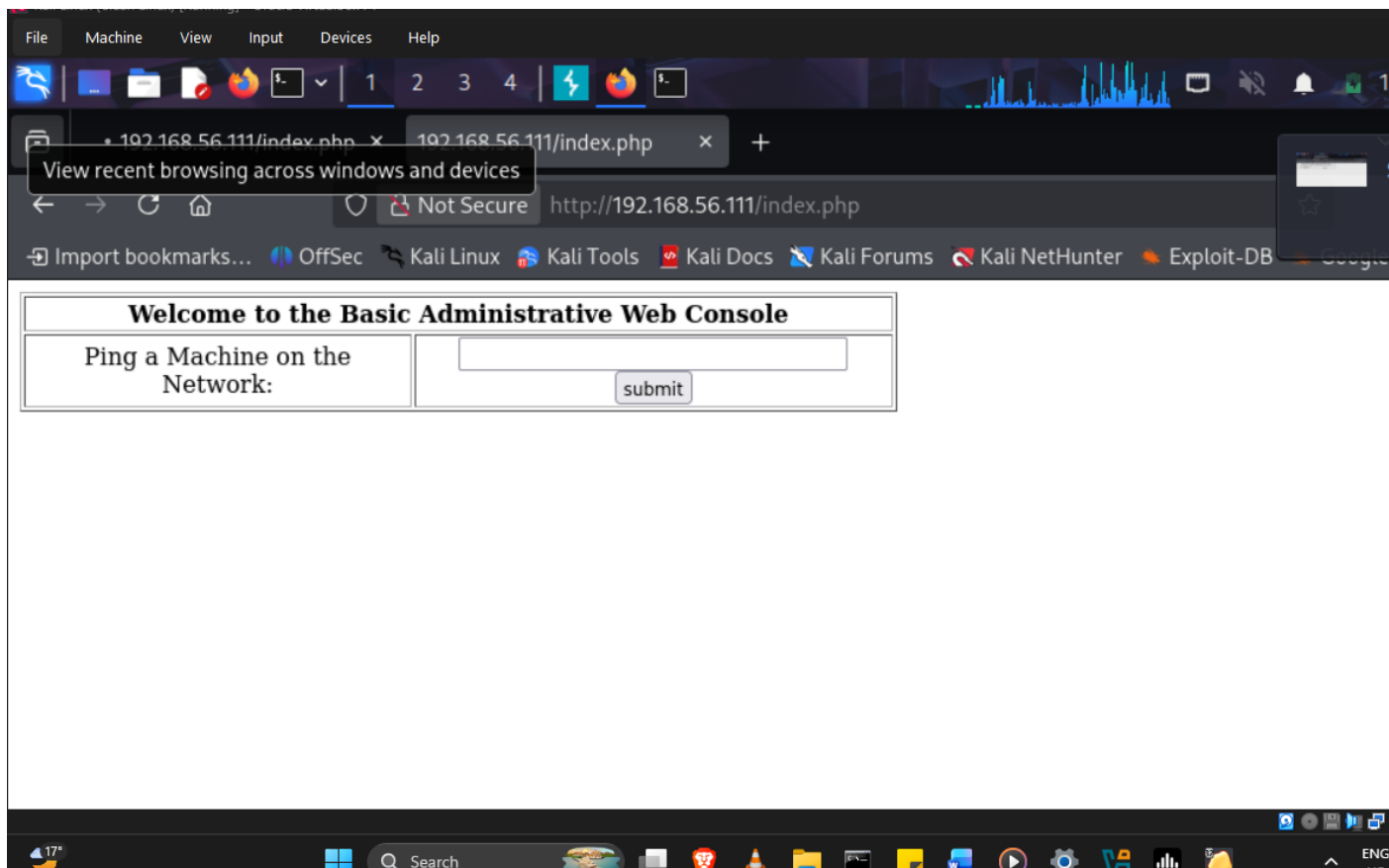
Privilege Escalation to Root

Local Environment Profiling

System architecture information gathered from the active target terminal identified a severely outdated kernel patch level:



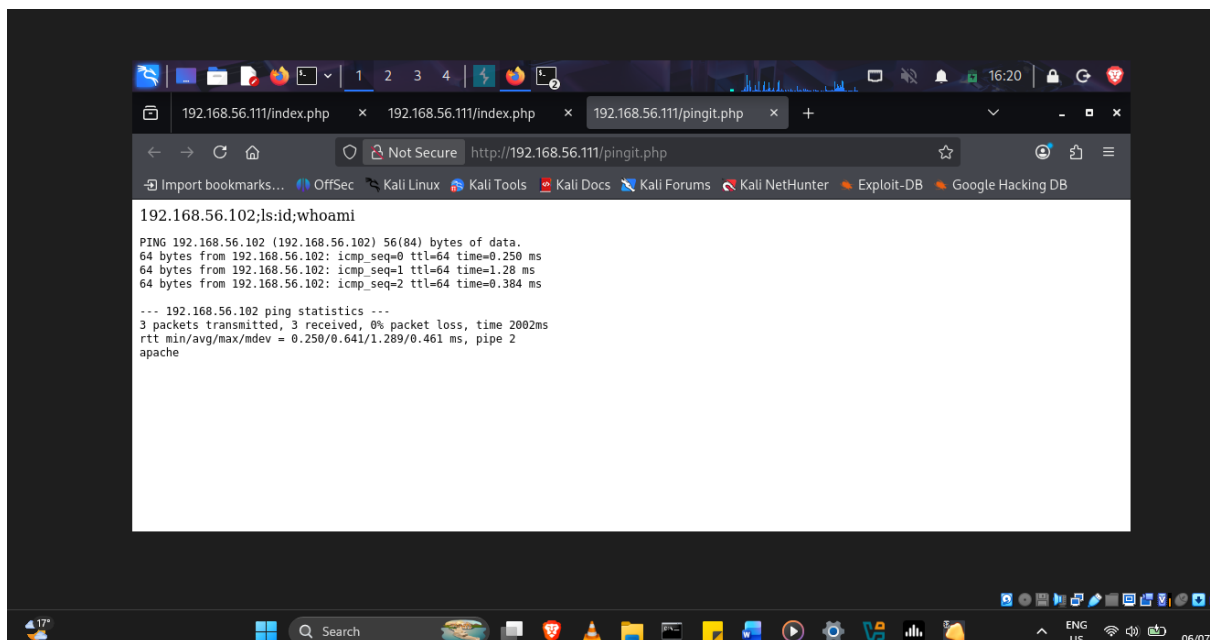
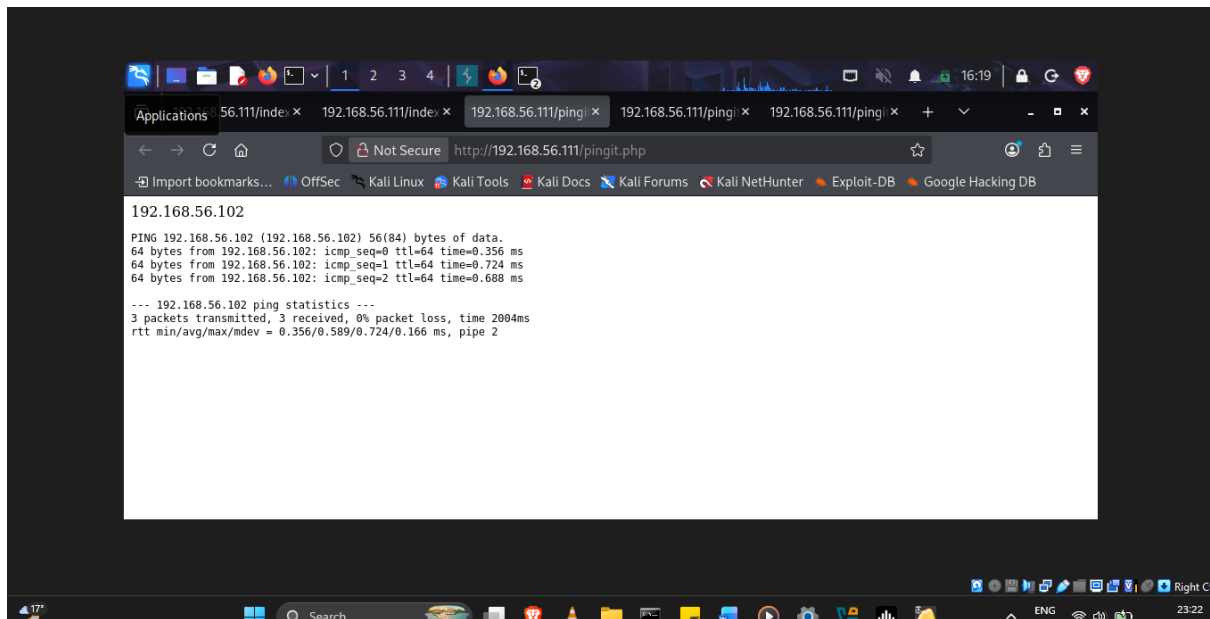
Local File Hosting: A quick temporary web server was initiated on the attacker machine inside the directory containing the exploit source code.



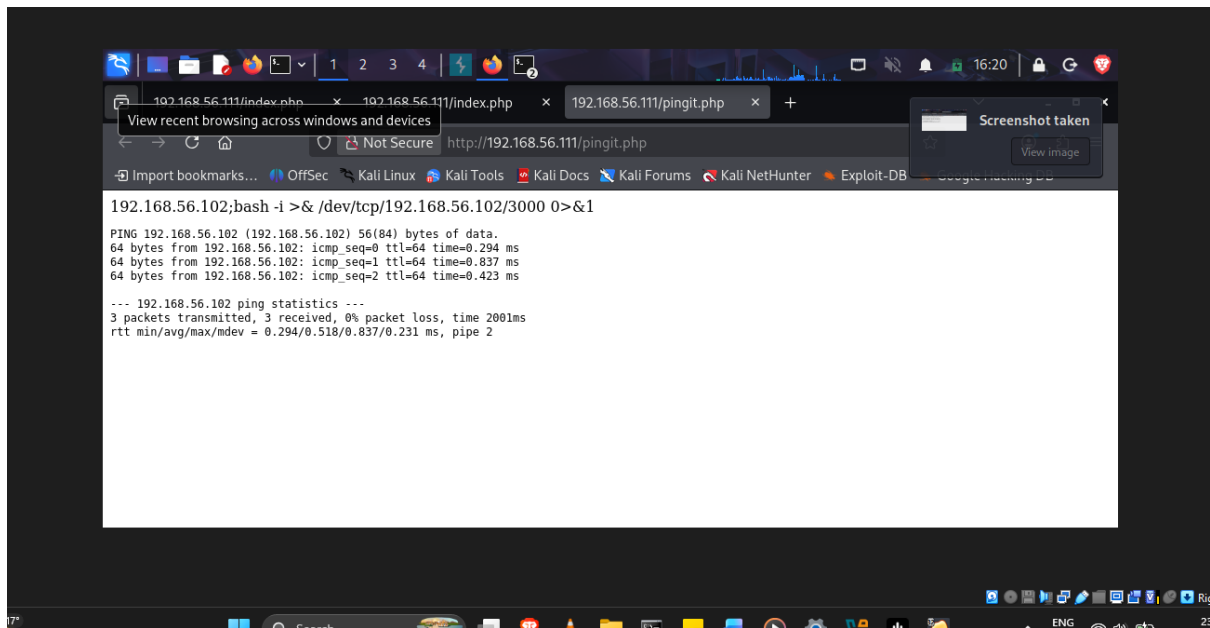
The console page displays the heading "Welcome to the Basic Administrative Web Console" along with a form labeled "Ping a Machine on the Network."

Submitting a standard IP address (192.168.56.102) returns the expected terminal output of a ping command.

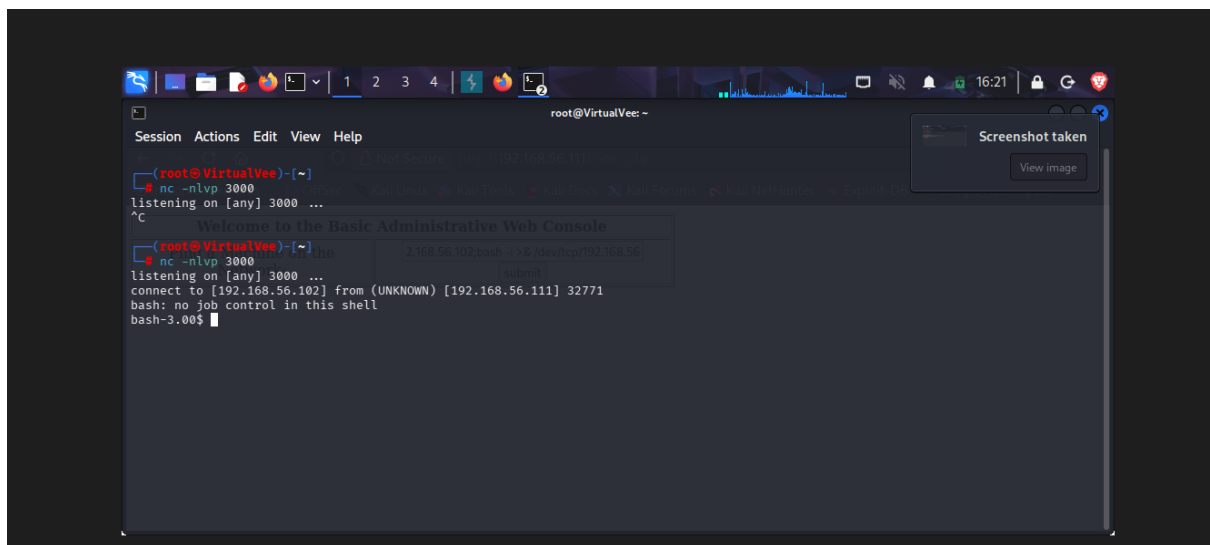
Appending a semicolon (;) instructs the target system to: *"first execute the ping, and then immediately execute the injected commands (ls, id, whoami)"*. The console output updates accordingly, displaying the word `apache` at the bottom, confirming code execution as the web server's runtime user.

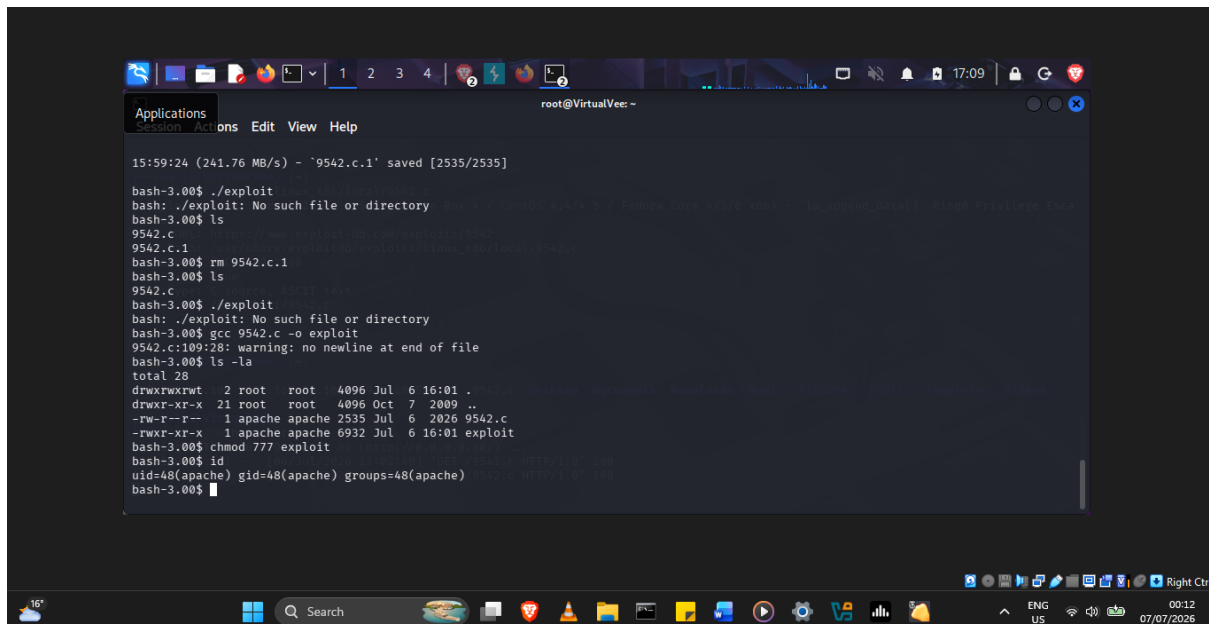


From the target system's open shell terminal, the source file was pulled onto the local machine using wget:



The exploit source code was compiled using gcc, permissions adjusted, and executed:



A screenshot of a terminal window titled 'root@VirtualVee: ~'. The terminal shows a series of commands and their outputs. The user starts by running './exploit', which results in 'bash: ./exploit: No such file or directory'. They then run 'ls', showing files '9542.c' and '9542.c.1'. Next, they run 'rm 9542.c.1'. After another 'ls', they run './exploit' again, resulting in 'bash: ./exploit: No such file or directory'. They then run 'gcc 9542.c -o exploit', which shows a warning: 'warning: no newline at end of file'. Finally, they run 'ls -la', showing a directory listing with permissions, owner, group, size, and date. The output shows that the user is now root (uid=0) and has full control of the system. The terminal window is part of a desktop environment with a taskbar at the bottom showing various application icons and system status indicators.

```
15:59:24 (241.76 MB/s) - '9542.c.1' saved [2535/2535]
bash-3.00$ ./exploit
bash: ./exploit: No such file or directory
bash-3.00$ ls
9542.c
9542.c.1
bash-3.00$ rm 9542.c.1
bash-3.00$ ls
9542.c
bash-3.00$ ./exploit
bash: ./exploit: No such file or directory
bash-3.00$ gcc 9542.c -o exploit
9542.c:109:28: warning: no newline at end of file
bash-3.00$ ls -la
total 28
drwxrwxrwt  2 root  root  4096 Jul  6 16:01 .
drwxr-xr-x 21 root  root  4096 Oct  7 2009 ..
-rw-r--r--  1 apache apache 2535 Jul  6 2026 9542.c
-rwxr-xr-x  1 apache apache 6932 Jul  6 16:01 exploit
bash-3.00$ chmod 777 exploit
bash-3.00$ id
uid=0(root) gid=48(apache) groups=48(apache)
bash-3.00$
```

Results

The exploit successfully bypassed legacy security bounds, granting instant administrative access with **root privileges** (uid=0), ensuring total control of the target virtual system.

Remediation Recommendations

- **Implement Input Sanitization:** Implement rigorous regular expression whitelisting on all administrative input fields (such as the ping tool) to ensure only clean, properly formatted IP addresses are passed to system commands.
- **Adopt Parameterized SQL Queries:** Re-engineer web login routines using parameterized inputs and prepared statements to mitigate authentication bypass via SQL injection text patterns.
- **System & Kernel Upgrades:** Decommission or upgrade legacy server environments running out-of-date Linux operating systems. Keeping operating system kernels up to date removes public, automated root exploitation scripts entirely

